

## Java 21 — Virtual Threads (Final)

**Java Version:** 21

**Year:** 2023

**Feature:** Virtual Threads — Final

**UniqueClassName:** VirtualThreads

### 1. Definition

**Virtual Threads** are lightweight threads introduced as a preview feature in Java 19 and finalized in **Java 21** as part of Project Loom.

Unlike traditional **platform threads**, which are closely associated with operating-system threads, virtual threads are managed by the **JVM** and are extremely lightweight.

They are designed especially for applications that perform large numbers of **I/O-bound tasks**, such as:

- HTTP requests
- Database queries
- File operations
- Network communication
- Microservice calls
- Message processing

A virtual thread can block on I/O without requiring one expensive OS thread per task. This allows Java applications to support very large numbers of concurrent tasks using a simple thread-per-request programming model.

### Platform Thread vs Virtual Thread

| Feature                        | Platform Thread       | Virtual Thread                  |
|--------------------------------|-----------------------|---------------------------------|
| Managed mainly by              | OS/JVM                | JVM                             |
| Memory overhead                | Relatively high       | Very low                        |
| Creation cost                  | Higher                | Very low                        |
| Suitable for millions of tasks | No                    | Yes, depending on workload      |
| Blocking I/O                   | Can consume OS thread | Usually releases carrier thread |
| Programming model              | Thread-per-task       | Thread-per-task                 |
| Best suited for                | CPU-bound work        | High-concurrency I/O            |

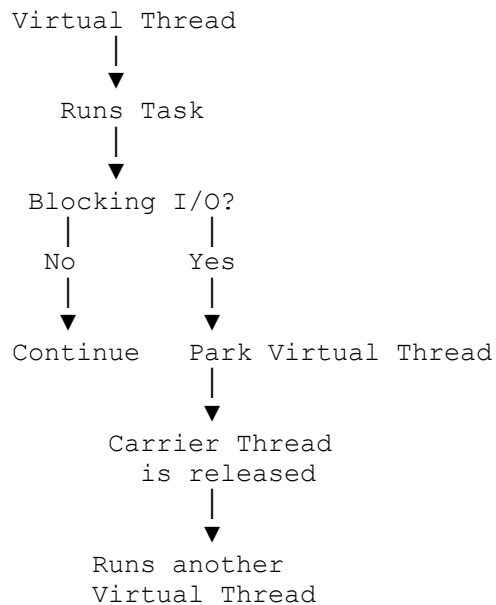
---

## 2. How Virtual Threads Work

A virtual thread runs on a real OS-backed **carrier thread**.

When the virtual thread performs a blocking operation, the JVM can **park** the virtual thread and allow the carrier thread to execute another virtual thread.

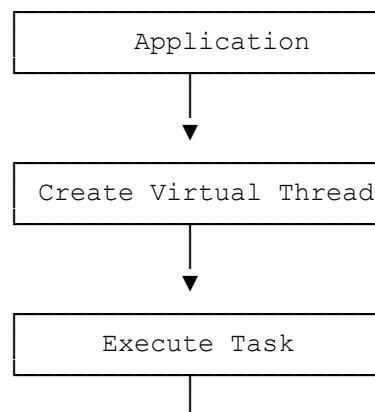
Conceptually:

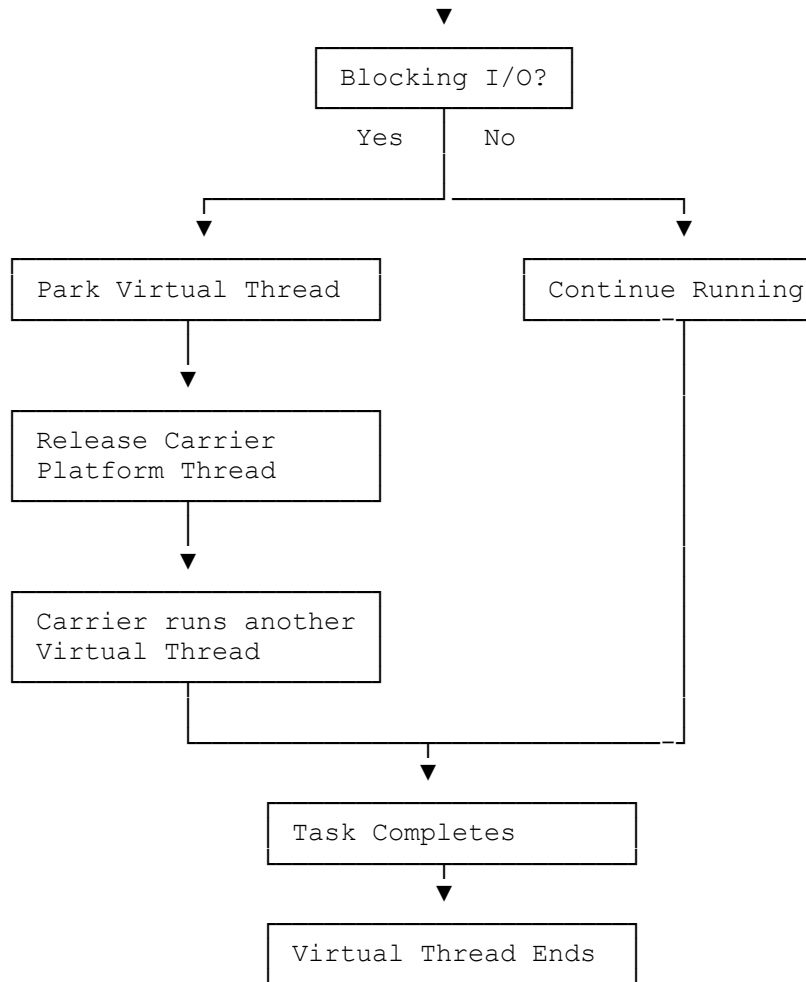


When the I/O operation becomes ready, the virtual thread can be scheduled again.

---

## 3. Flowchart





---

## 4. Java Code Example

### Basic Virtual Thread

```
public class VirtualThreads {  
  
    public static void main(String[] args) throws Exception {  
  
        Thread virtualThread = Thread.startVirtualThread(() -> {  
  
            System.out.println(  
                "Running in: " + Thread.currentThread()  
            );  
  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
  
        })  
  
    }  
}
```

```

        System.out.println("Task completed");
    });

    virtualThread.join();
}
}

```

## Example Output

Running in: VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1  
Task completed

The exact thread identifier and carrier-thread details can vary.

---

## 5. Creating Many Virtual Threads

One of the major advantages is that creating a very large number of concurrent tasks is practical.

```

public class VirtualThreads {

    public static void main(String[] args) throws Exception {

        for (int i = 1; i <= 10_000; i++) {

            int taskId = i;

            Thread.startVirtualThread(() -> {

                try {
                    Thread.sleep(1000);

                    System.out.println(
                        "Task " + taskId +
                        " executed by " +
                        Thread.currentThread()
                    );

                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

            });

            Thread.sleep(3000);

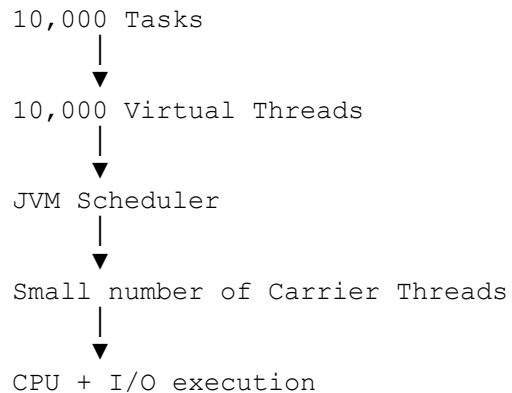
            System.out.println("All tasks submitted.");

        }

    }
}

```

This demonstrates the intended programming model:



---

## 6. Recommended Executor Usage

For application code, `Executors.newVirtualThreadPerTaskExecutor()` is often convenient:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class VirtualThreads {

    public static void main(String[] args) {

        try (ExecutorService executor =
            Executors.newVirtualThreadPerTaskExecutor()) {

            for (int i = 1; i <= 10; i++) {

                int taskId = i;

                executor.submit(() -> {

                    System.out.println(
                        "Task " + taskId +
                        " running on " +
                        Thread.currentThread()
                    );

                    Thread.sleep(1000);

                    return null;
                });
            }

            System.out.println("All tasks completed.");
        }
    }
}
```

The important API is:

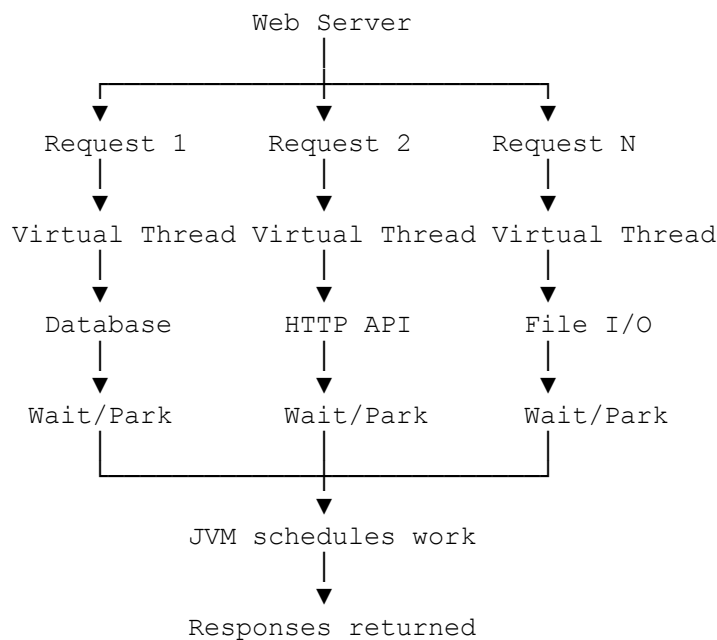
`Executors.newVirtualThreadPerTaskExecutor()`

It creates a new virtual thread for each submitted task rather than maintaining a small fixed pool of platform threads.

---

## 7. Real-World Example

Consider a web server receiving 100,000 requests:



This is particularly useful for **I/O-heavy server applications**, because developers can write straightforward blocking-style code without needing to transform everything into asynchronous callback-based code.

---

## 8. Important Limitation

Virtual threads **do not make CPU-intensive work faster**.

For example:

```
Thread.startVirtualThread(() -> {  
    performHugeCpuCalculation();  
});
```

If the task continuously consumes CPU, virtual threads still need CPU time from the available processors.

Therefore:

I/O-bound workload  
↓  
Virtual Threads  
↓  
Excellent scalability

but:

CPU-bound workload  
↓  
Virtual Threads  
↓  
Does NOT create additional CPU capacity

---

## Key Takeaway

**Java 21 finalized Virtual Threads**, providing a lightweight concurrency model that allows developers to create huge numbers of threads for concurrent, especially I/O-bound, tasks.

The central idea is:

Traditional:

Request → Platform Thread → Blocking I/O  
↓  
OS thread waits

Virtual Threads:

Request → Virtual Thread → Blocking I/O  
↓  
Virtual thread parks  
↓  
Carrier reused  
↓  
Another task executes

**In one sentence:**

**Virtual threads let Java applications use a simple thread-per-task programming model with dramatically lower thread overhead, making high-concurrency I/O workloads much easier to scale.**

## Java 21 — Pattern Matching for switch (Final)

**Java Version:** 21

**Year:** 2023

**Feature:** Pattern Matching for switch — Final

**UniqueClassName:** PatternMatchingSwitch

### 1. Definition

**Pattern Matching for switch** was finalized in **Java 21**. It extends the traditional switch statement/expression so that cases can match **types and patterns**, not just constant values.

It works particularly well with **sealed classes, records, and type hierarchies**, allowing developers to replace long if-else instanceof chains with concise and readable switch logic.

### Traditional approach

```
if (value instanceof Integer) {  
    return "Integer: " + value;  
} else if (value instanceof String) {  
    return "String: " + value;  
} else {  
    return "Other";  
}
```

### Java 21 approach

```
return switch (value) {  
    case Integer i -> "Integer: " + i;  
    case String s  -> "String: " + s;  
    default        -> "Other";  
};
```

The major concept is:

Traditional switch

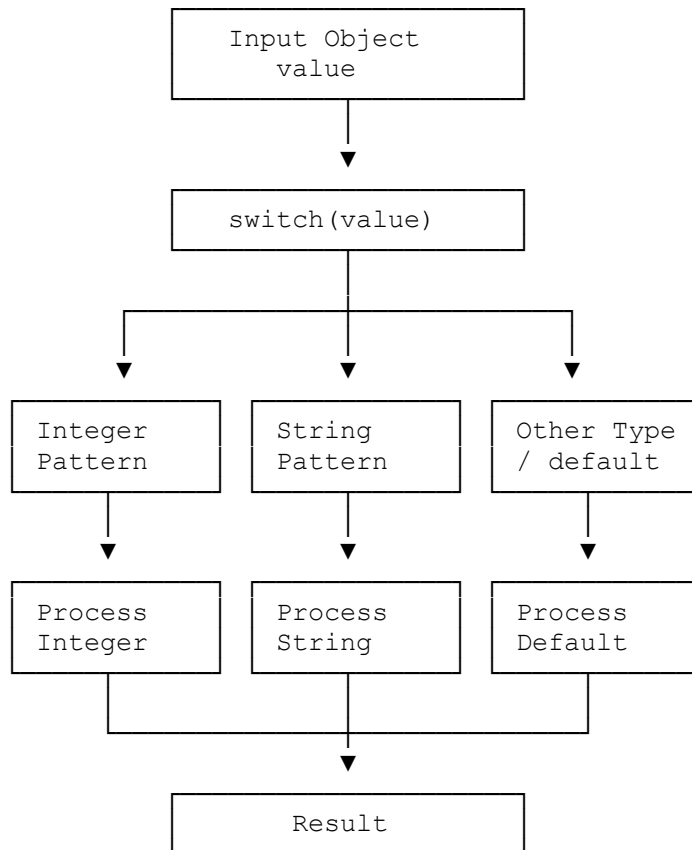
```
|  
├─ constant matching  
└─ enum matching
```

## Java 21 Pattern Matching for switch

- constant matching
- type matching
- guarded matching with `when`
- record/deconstruction patterns

---

## 2. Flowchart



---

## 3. Basic Java Code Example

```
public class PatternMatchingSwitch {  
    static String describe(Object value) {  
        return switch (value) {  
            case Integer i ->  
                "Integer: " + i;  
            case String s ->
```

```

        "String: " + s;

    case Double d ->
        "Double: " + d;

    case Boolean b ->
        "Boolean: " + b;

    default ->
        "Unknown type";
    };
}

public static void main(String[] args) {

    System.out.println(describe(100));
    System.out.println(describe("Java"));
    System.out.println(describe(25.5));
    System.out.println(describe(true));
}
}

```

## Output

```

Integer   : 100
String    : Java
Double    : 25.5
Boolean   : true

```

Notice that the variable is declared directly inside the pattern:

```
case Integer i
```

Here:

- Integer → type being matched
- i → variable bound to the matched value

---

## 4. Pattern Matching with when

Java 21 also supports conditional pattern matching using a **when guard**.

```

public class PatternMatchingSwitch {

    static String classify(Object value) {

        return switch (value) {

            case Integer i when i > 0 ->

```

```

        "Positive integer";

    case Integer i when i < 0 ->
        "Negative integer";

    case Integer i ->
        "Zero";

    case String s when s.isEmpty() ->
        "Empty string";

    case String s ->
        "Non-empty string";

    default ->
        "Other";
};

}

public static void main(String[] args) {

    System.out.println(classify(10));
    System.out.println(classify(-5));
    System.out.println(classify(0));
    System.out.println(classify(""));
    System.out.println(classify("Java"));

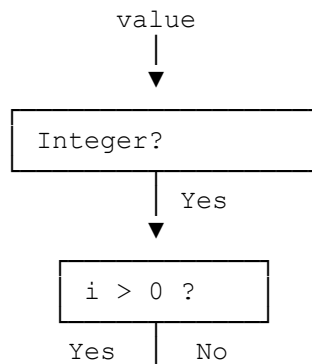
}
}

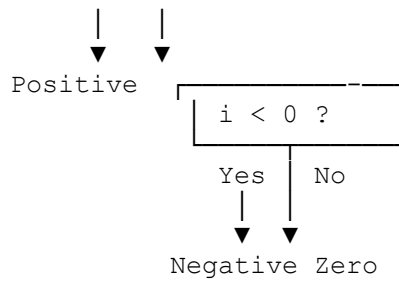
```

## Output

Positive integer  
 Negative integer  
 Zero  
 Empty string  
 Non-empty string

The execution can be visualized as:





---

## 5. Pattern Matching with null

Traditional switch handling of null can be inconvenient. Pattern matching for switch allows null to be handled explicitly.

```
public class PatternMatchingSwitch {  
    static String describe(Object value) {  
        return switch (value) {  
            case null ->  
                "Value is null";  
  
            case String s ->  
                "String: " + s;  
  
            case Integer i ->  
                "Integer: " + i;  
  
            default ->  
                "Other";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(describe(null));  
        System.out.println(describe("Java"));  
        System.out.println(describe(21));  
    }  
}
```

Output:

```
Value is null  
String    : Java  
Integer   : 21
```

---

## 6. Pattern Matching with Sealed Classes

This feature becomes particularly powerful when combined with **sealed classes**.

```
sealed interface Shape
    permits Circle, Rectangle, Square {
}

record Circle(double radius) implements Shape {
}

record Rectangle(double width, double height) implements Shape {
}

record Square(double side) implements Shape {
}
```

Now we can switch over all permitted types:

```
public class PatternMatchingSwitch {

    static double area(Shape shape) {

        return switch (shape) {

            case Circle c ->
                Math.PI * c.radius() * c.radius();

            case Rectangle r ->
                r.width() * r.height();

            case Square s ->
                s.side() * s.side();

        };
    }

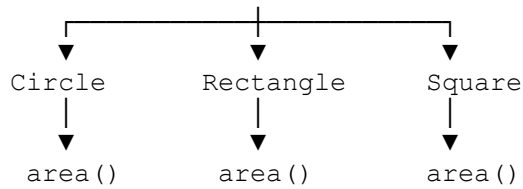
    public static void main(String[] args) {

        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(4, 6);
        Shape square = new Square(3);

        System.out.println(area(circle));
        System.out.println(area(rectangle));
        System.out.println(area(square));
    }
}
```

Here, a default case is unnecessary because the compiler knows that Shape can only be one of the permitted types.

```
Shape
|
```



---

## 7. Why Pattern Matching for switch Is Useful

### Before Java 21

```
if (obj instanceof String s) {  
    // ...  
} else if (obj instanceof Integer i) {  
    // ...  
} else if (obj instanceof Double d) {  
    // ...  
} else {  
    // ...  
}
```

### Java 21

```
return switch (obj) {  
    case String s -> "...";  
    case Integer i -> "...";  
    case Double d -> "...";  
    default -> "...";  
};
```

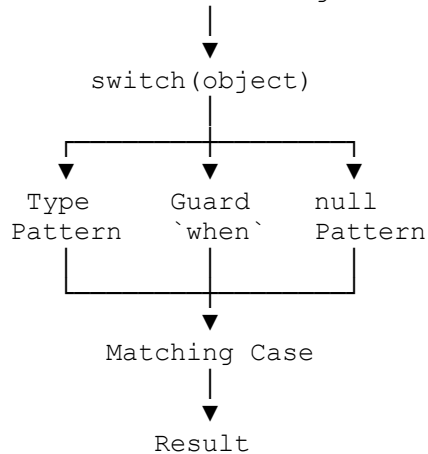
### Benefits

- **More readable** type-based branching
- Less repetitive instanceof code
- Works naturally with **switch expressions**
- Supports **guards using when**
- Can explicitly handle **null**
- Works very well with **sealed hierarchies**
- Compiler can perform **exhaustiveness checking**
- Particularly useful with **records and record patterns**

### Key Takeaway

Java 21 finalized Pattern Matching for `switch`, allowing `switch` to match objects by type and apply conditions directly, making complex type-based branching shorter, safer, and more expressive.

Java 21 Pattern Matching for switch



## Java 21 — Record Patterns (Final)

**Java Version:** 21

**Year:** 2023

**Feature:** Record Patterns — Final

**UniqueClassName:** RecordPatterns

### 1. Definition

**Record Patterns** were finalized in **Java 21**. They allow a Java program to **deconstruct a record directly into its component values** during pattern matching.

In simpler terms, instead of first checking whether an object is a particular record and then calling its accessor methods manually, a record pattern lets you **match the record and extract its components in one step**.

#### Without Record Patterns

```
if (obj instanceof Point point) {  
    int x = point.x();  
    int y = point.y();  
  
    System.out.println(x + ", " + y);  
}
```

#### With Record Patterns

```
if (obj instanceof Point(int x, int y)) {
```

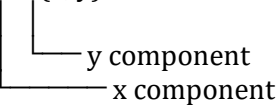
```
    System.out.println(x + ", " + y);  
}
```

The record:

```
record Point(int x, int y) {  
}
```

can therefore be **deconstructed** as:

Point(x, y)



---

## 2. Basic Concept

A record pattern has this general form:

```
RecordType(Type1 variable1, Type2 variable2)
```

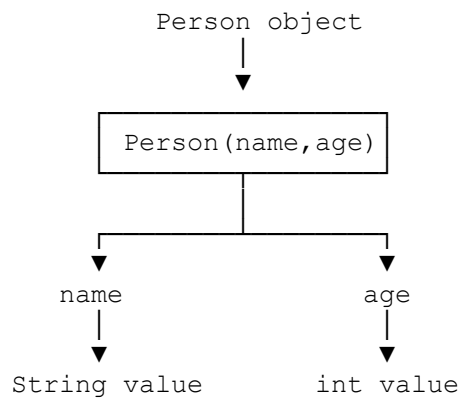
For example:

```
record Person(String name, int age) {  
}
```

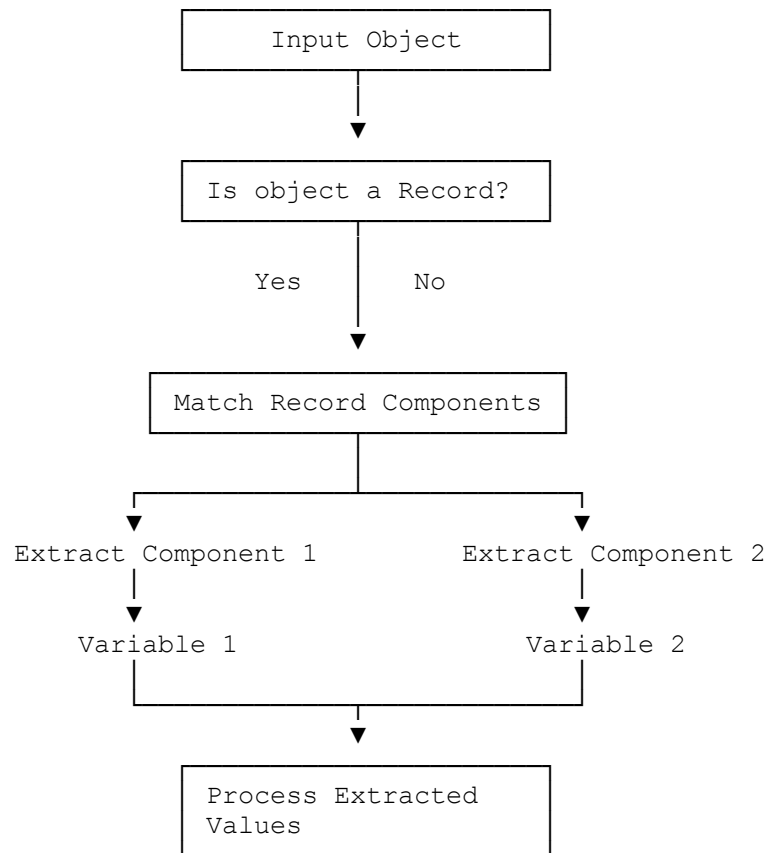
can be matched using:

```
Person(String name, int age)
```

Conceptually:



### 3. Flowchart



---

### 4. Basic Java Code Example

```
public class RecordPatterns {  
  
    record Point(int x, int y) {  
    }  
  
    public static void main(String[] args) {  
  
        Object obj = new Point(10, 20);  
  
        if (obj instanceof Point(int x, int y)) {  
  
            System.out.println("X = " + x);  
            System.out.println("Y = " + y);  
        }  
    }  
}
```

## Output

X = 10

Y = 20

The following pattern:

Point(int x, int y)

does two things:

1. Checks that the object is a Point.
2. Extracts its x and y components.

---

## 5. Record Patterns with switch

Record patterns become particularly powerful when combined with **Pattern Matching for switch**.

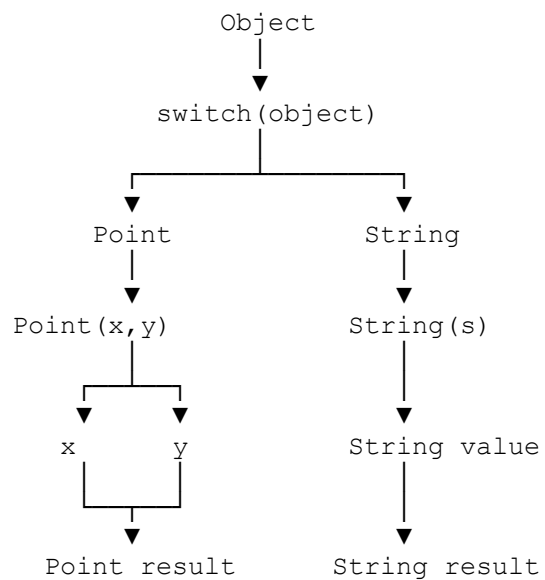
```
public class RecordPatterns {  
    record Point(int x, int y) {  
    }  
  
    static String describe(Object obj) {  
        return switch (obj) {  
            case Point(int x, int y) ->  
                "Point: x=" + x + ", y=" + y;  
            case String s ->  
                "String: " + s;  
            default ->  
                "Unknown object";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(  
            describe(new Point(10, 20))  
        );  
  
        System.out.println(  
            describe("Java")  
        );  
    }  
}
```

## Output

Point : x=10,y=20

String : Java

Flow:



---

## 6. Nested Record Patterns

One of the most useful capabilities is **nested deconstruction**.

Consider:

```
record Address(String city, String country) {  
}
```

```
record Person(String name, Address address) {  
}
```

We can deconstruct both records:

```
public class RecordPatterns {  
    record Address(String city, String country) {  
    }  
  
    record Person(String name, Address address) {  
    }  
}
```

```

public static void main(String[] args) {

    Person person =
        new Person(
            "Anurag",
            new Address("Bilaspur", "India")
        );

    if (person instanceof Person(
        String name,
        Address(String city, String country)
    )) {

        System.out.println("Name: " + name);
        System.out.println("City: " + city);
        System.out.println("Country: " + country);
    }
}

```

## Output

Name: Anurag  
 City: Bilaspur  
 Country: India

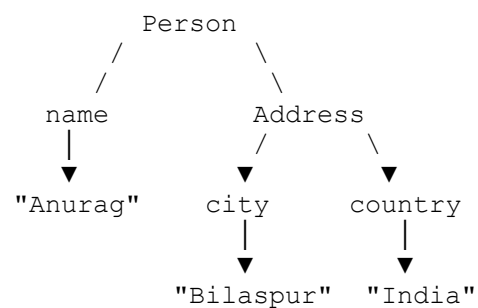
The nested pattern:

```

Person(
    String name,
    Address(String city, String country)
)

```

can be visualized as:



## 7. Record Patterns with Conditions

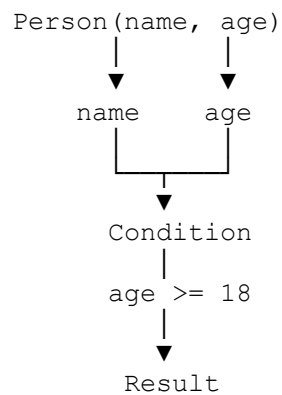
Record patterns can also be combined with conditions.

```
public class RecordPatterns {  
    record Person(String name, int age) {  
    }  
  
    static String classify(Person person) {  
        if (person instanceof Person(String name, int age)  
            && age >= 18) {  
            return name + " is an adult";  
        }  
        return "Person is a minor";  
    }  
  
    public static void main(String[] args) {  
        Person person = new Person("Rahul", 25);  
        System.out.println(classify(person));  
    }  
}
```

### Output

Rahul is an adult

The pattern extracts the values:



## 8. Record Patterns + Pattern Matching switch

These two Java 21 features work very well together.

```
public class RecordPatterns {  
    record Point(int x, int y) {  
    }  
  
    static String classify(Point point) {  
        return switch (point) {  
            case Point(0, 0) ->  
                "Origin";  
  
            case Point(int x, int y) when x == 0 ->  
                "On Y-axis";  
  
            case Point(int x, int y) when y == 0 ->  
                "On X-axis";  
  
            case Point(int x, int y) ->  
                "Point at (" + x + ", " + y + ")";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(classify(new Point(0, 0)));  
        System.out.println(classify(new Point(0, 10)));  
        System.out.println(classify(new Point(20, 0)));  
        System.out.println(classify(new Point(10, 20)));  
    }  
}
```

### Output

Origin  
On Y-axis  
On X-axis  
Point at (10, 20)

---

## 9. Traditional vs Record Pattern

### Traditional Java

```
if (obj instanceof Point point) {  
    int x = point.x();  
}
```

```

    int y = point.y();

    System.out.println(x + ", " + y);
}

```

## Java 21

```

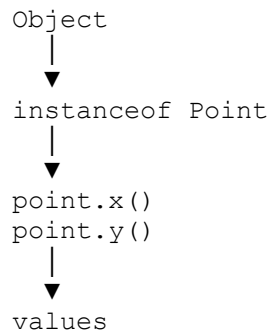
if (obj instanceof Point(int x, int y)) {

    System.out.println(x + ", " + y);
}

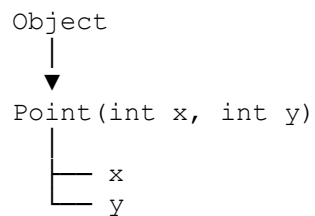
```

The Java 21 version eliminates explicit accessor calls:

Traditional



Record Pattern



## 10. Key Benefits

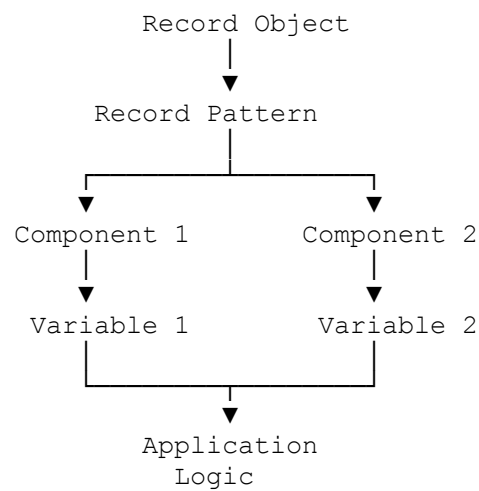
| Benefit                 | Description                                       |
|-------------------------|---------------------------------------------------|
| <b>Deconstruction</b>   | Extract record components directly                |
| <b>Less boilerplate</b> | No need for repeated accessor calls               |
| <b>Readable</b>         | Structure of the record is visible in the pattern |
| <b>Type-safe</b>        | Compiler verifies pattern types                   |

|                              |                                                     |
|------------------------------|-----------------------------------------------------|
|                              |                                                     |
| <b>Nested matching</b>       | Records can be deconstructed recursively            |
| <b>Works with switch</b>     | Excellent combination with Java 21 pattern matching |
| <b>Works with instanceof</b> | Simplifies type checking and extraction             |

## Key Takeaway

**Record Patterns in Java 21 allow records to be deconstructed directly inside pattern matching expressions.**

The core idea is:



In one sentence:

**Record patterns make it possible to match a record and extract its components in a single, concise pattern, especially simplifying code when combined with instanceof and switch.**

## Java 21 — Sequenced Collections

**Java Version:** 21

**Year:** 2023

**Feature:** Sequenced Collections

**UniqueClassName:** SequencedCollections

### 1. Definition

**Sequenced Collections** were introduced in **Java 21** through a new set of interfaces that provide a **uniform way to work with collections that have a defined encounter order**.

Before Java 21, operations such as "get the first element," "get the last element," "add at the beginning," or "reverse the collection" often differed between collection types.

Java 21 introduces:

- SequencedCollection
- SequencedSet
- SequencedMap

The main idea is to provide a common API for working with the **first and last elements** of an ordered collection.

### Important methods

For SequencedCollection:

```
getFirst()
getLast()
addFirst()
addLast()
removeFirst()
removeLast()
reversed()
```

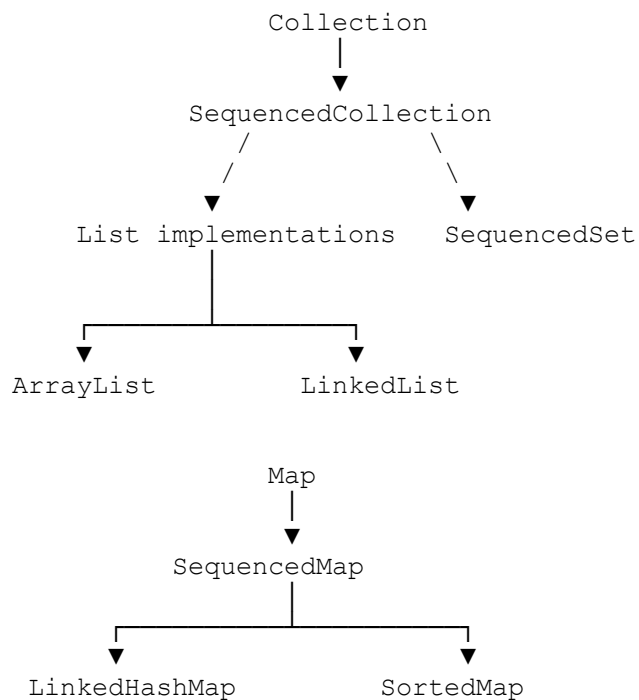
For SequencedMap:

```
firstEntry()
lastEntry()
putFirst()
putLast()
pollFirstEntry()
pollLastEntry()
reversed()
```

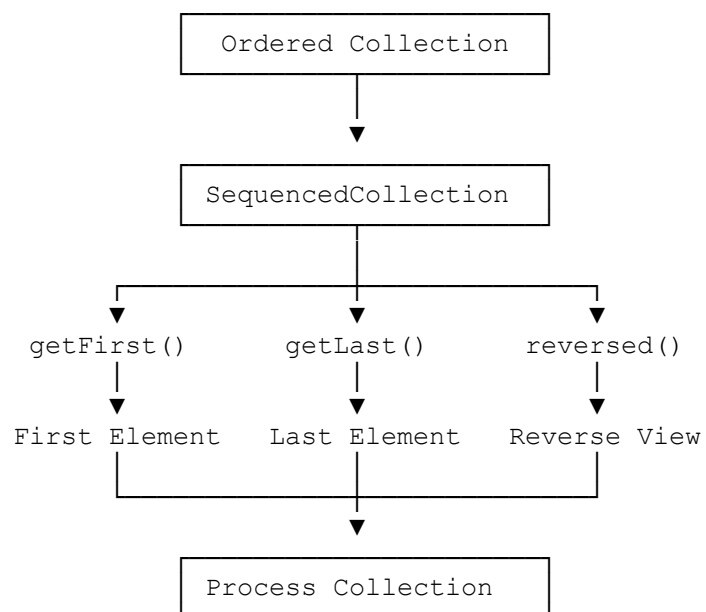
---

## 2. Collection Hierarchy

The new interfaces fit into the existing Java Collections Framework:



## 3. Flowchart



## 4. Java Code Example — SequencedCollection

```
import java.util.ArrayList;
import java.util.SequencedCollection;

public class SequencedCollections {

    public static void main(String[] args) {

        SequencedCollection<String> languages =
            new ArrayList<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("Go");

        System.out.println("Collection: " + languages);

        System.out.println("First: "
            + languages.getFirst());

        System.out.println("Last: "
            + languages.getLast());

        languages.addFirst("C++");
        languages.addLast("Rust");

        System.out.println("After adding:");
        System.out.println(languages);

        languages.removeFirst();
        languages.removeLast();

        System.out.println("After removing:");
        System.out.println(languages);

        System.out.println("Reversed: "
            + languages.reversed());
    }
}
```

### Output

```
Collection      : [Java, Python, Go]
First           : Java
Last            : Go
```

```
After adding:
[C++, Java, Python, Go, Rust]
```

```
After removing:
[Java, Python, Go]
```

Reversed: [Go, Python, Java]

---

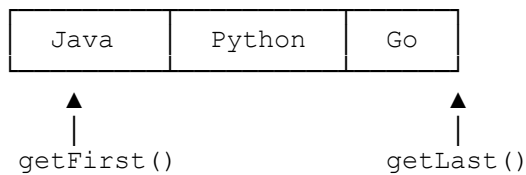
## 5. Understanding the New Methods

Suppose we have:

```
SequencedCollection<String> languages =  
    new ArrayList<>();
```

```
languages.add("Java");  
languages.add("Python");  
languages.add("Go");
```

The collection is:



### Add at beginning

```
languages.addFirst("C++");
```

Result:

[C++, Java, Python, Go]

### Add at end

```
languages.addLast("Rust");
```

Result:

[C++, Java, Python, Go, Rust]

### Remove first

```
languages.removeFirst();
```

Result:

[Java, Python, Go, Rust]

## Remove last

```
languages.removeLast();
```

Result:

```
[Java, Python, Go]
```

---

## 6. reversed() Method

One particularly useful feature is:

```
languages.reversed()
```

For:

```
[Java, Python, Go]
```

the reversed view represents:

```
[Go, Python, Java]
```

Example:

```
import java.util.ArrayList;
import java.util.SequencedCollection;

public class SequencedCollections {

    public static void main(String[] args) {

        SequencedCollection<Integer> numbers =
            new ArrayList<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);

        SequencedCollection<Integer> reversed =
            numbers.reversed();

        System.out.println("Original: " + numbers);
        System.out.println("Reversed: " + reversed);
    }
}
```

Output:

Original : [10, 20, 30, 40]  
Reversed : [40, 30, 20, 10]

reversed() provides a **reverse-ordered view**, rather than requiring you to create a separate copied collection.

---

## 7. Sequenced Maps

Java 21 also introduces SequencedMap.

```
import java.util.LinkedHashMap;
import java.util.SequencedMap;

public class SequencedCollections {

    public static void main(String[] args) {

        SequencedMap<Integer, String> students =
            new LinkedHashMap<>();

        students.put(1, "Rahul");
        students.put(2, "Priya");
        students.put(3, "Amit");

        System.out.println("Map: " + students);

        System.out.println(
            "First Entry: " + students.firstEntry()
        );

        System.out.println(
            "Last Entry: " + students.lastEntry()
        );

        System.out.println(
            "Reversed: " + students.reversed()
        );
    }
}
```

### Output:

```
Map: {1=Rahul, 2=Priya, 3=Amit}
First Entry: 1=Rahul
Last Entry: 3=Amit
Reversed: {3=Amit, 2=Priya, 1=Rahul}
```

---

## 8. Why Sequenced Collections Were Introduced

Before Java 21, different collection types often required different techniques.

For example, obtaining the last element of a List could involve:

```
list.get(list.size() - 1);
```

Java 21 provides:

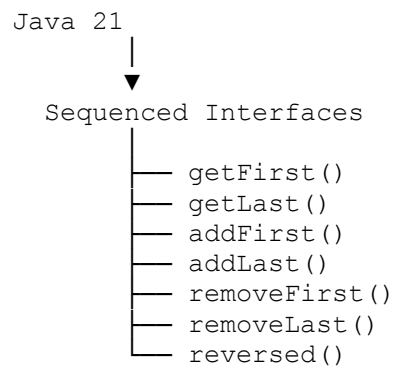
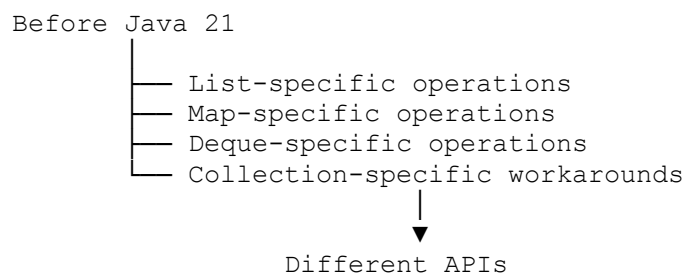
```
list.getLast();
```

Similarly, adding to the beginning becomes:

```
list.addFirst(value);
```

instead of relying on collection-specific operations.

The goal is **API consistency**:



## 9. Main Interfaces

| Interface                           | Purpose                                       |
|-------------------------------------|-----------------------------------------------|
| <b>SequencedCollection&lt;E&gt;</b> | Ordered collection with first/last operations |
| <b>SequencedSet&lt;E&gt;</b>        | Ordered set with first/last operations        |
| <b>SequencedMap&lt;K,V&gt;</b>      | Ordered map with first/last entries           |

### Core collection API

```
getFirst()
getLast()
addFirst(e)
addLast(e)
removeFirst()
removeLast()
reversed()
```

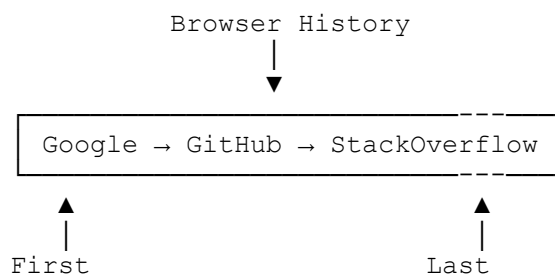
### Core map API

```
firstEntry()
lastEntry()
putFirst(k, v)
putLast(k, v)
pollFirstEntry()
pollLastEntry()
reversed()
```

---

## 10. Real-World Example

A browser history is naturally sequenced:



With Java 21:

```
SequencedCollection<String> history =  
    new ArrayList<>();
```

```
history.add("Google");  
history.add("GitHub");  
history.add("StackOverflow");
```

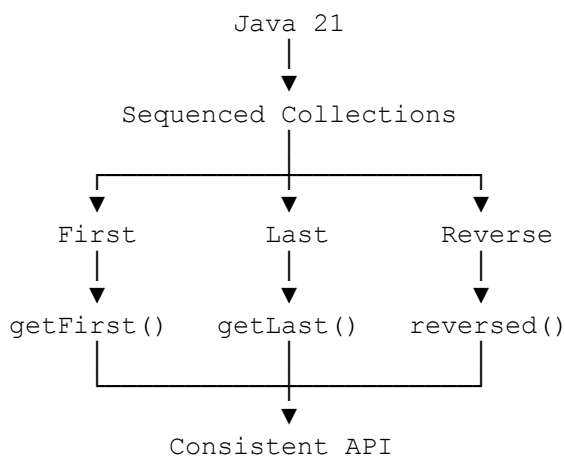
```
System.out.println(history.getFirst());  
System.out.println(history.getLast());
```

This provides a consistent API for applications where **encounter order matters**.

### Key Takeaway

**Sequenced Collections in Java 21 standardize operations on collections and maps that have a defined encounter order.**

The central concept is:



**In one sentence:**

**Sequenced Collections make it easier and more consistent to access, add, remove, and reverse elements at the beginning and end of ordered collections and maps.**

## Java 21 — Foreign Function and Memory API (Final)

**Java Version:** 21

**Year:** 2023

**Feature:** Foreign Function and Memory API (FFM API) — Final

**UniqueClassName:** FFM\_API

### 1. Definition

The **Foreign Function and Memory API (FFM API)** was finalized in **Java 21**. It provides a standard Java API for interacting with:

1. **Foreign memory** — memory outside the Java heap.
2. **Foreign functions** — functions implemented outside the JVM, such as native C functions.

It provides a safer and more modern alternative to older mechanisms such as **JNI** and the `sun.misc.Unsafe`-based approaches.

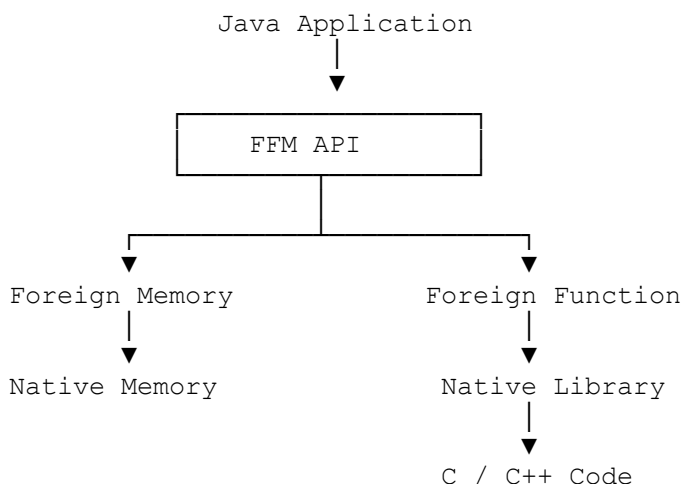
The API is primarily based around:

- **Arena** — controls the lifetime of native memory.
- **MemorySegment** — represents a region of memory.
- **MemoryLayout** — describes the structure/layout of memory.
- **Linker** — connects Java code to foreign/native functions.
- **FunctionDescriptor** — describes the arguments and return type of a foreign function.
- **SymbolLookup** — locates native functions.

---

### 2. Basic Concept

The FFM API creates a bridge between Java and native code:



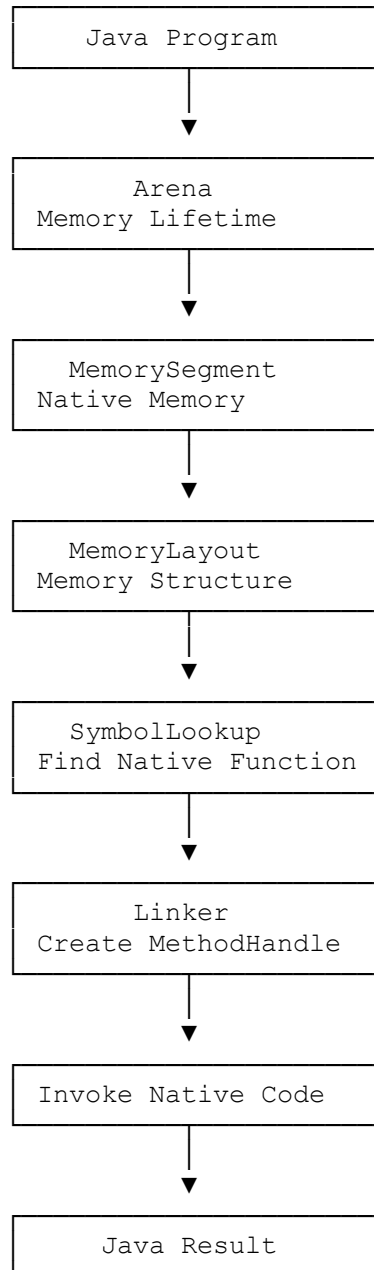
For example, Java can call a native C function such as:

```
int strlen(const char* str);
```

without writing a traditional JNI wrapper.

---

### 3. Flowchart



## 4. Important FFM API Components

| Component                 | Purpose                                            |
|---------------------------|----------------------------------------------------|
| <b>Arena</b>              | Controls allocation and lifetime of foreign memory |
| <b>MemorySegment</b>      | Represents a memory region                         |
| <b>MemoryLayout</b>       | Describes memory organization                      |
| <b>ValueLayout</b>        | Describes primitive values such as JAVA_INT        |
| <b>Linker</b>             | Provides access to foreign functions               |
| <b>SymbolLookup</b>       | Finds native function symbols                      |
| <b>FunctionDescriptor</b> | Describes function parameters and return type      |

---

## 5. Java Code Example — Native strlen

One classic demonstration is calling the native C library function `strlen()`.

On many Unix-like systems, `strlen` is available through the standard C library.

```
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.SymbolLookup;
import java.lang.invoke.MethodHandle;

import static java.lang.foreign.ValueLayout.*;

public class FFM_API {

    public static void main(String[] args) throws Throwable {

        Linker linker = Linker.nativeLinker();

        SymbolLookup stdlib = linker.defaultLookup();

        MemorySegment strlenAddress =
            stdlib.find("strlen")
                .orElseThrow();

        MethodHandle strlen = linker.downcallHandle(
            strlenAddress,
            FunctionDescriptor.of(
```

```

        JAVA_LONG,
        ADDRESS
    )
);

try (Arena arena = Arena.ofConfined()) {
    String text = "Hello Java 21";

    MemorySegment nativeString =
        arena.allocateFrom(text);

    long length =
        (long) strlen.invoke(nativeString);

    System.out.println(
        "String: " + text
    );

    System.out.println(
        "Length: " + length
    );
}
}
}

```

## Output

```

String   : Hello Java 21
Length   : 13

```

The exact native ABI details can vary across platforms, so native interoperation should be tested on the target operating system/JDK.

---

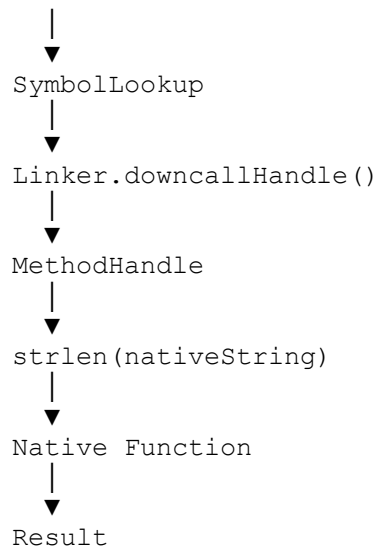
## 6. How This Example Works

The important sequence is:

```

String
|
▼
Arena.allocateFrom()
|
▼
MemorySegment
|
▼
Native Memory
|
▼
Find "strlen"

```



## 7. Foreign Memory Example

FFM is not only about calling native functions. It also provides APIs for working with memory outside the Java heap.

```
import java.lang.foreign.Arena;
import java.lang.foreign.MemorySegment;

import static java.lang.foreign.ValueLayout.JAVA_INT;

public class FFM_API {

    public static void main(String[] args) {

        try (Arena arena = Arena.ofConfined()) {

            MemorySegment memory =
                arena.allocate(JAVA_INT);

            memory.set(JAVA_INT, 0, 42);

            int value =
                memory.get(JAVA_INT, 0);

            System.out.println(
                "Value = " + value
            );

        }

    }

}
```

## Output

Value = 42

Here:

```
MemorySegment memory = arena.allocate(JAVA_INT);
```

allocates native/foreign memory capable of storing an integer.

Then:

```
memory.set(JAVA_INT, 0, 42);
```

writes 42 into that memory.

And:

```
memory.get(JAVA_INT, 0);
```

reads the value back.

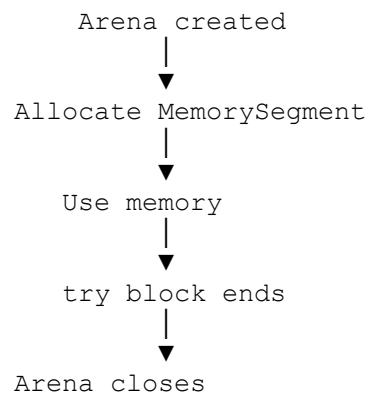
---

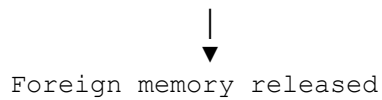
## 8. Arena and Memory Lifetime

One of the important safety features of the FFM API is explicit memory lifetime management.

```
try (Arena arena = Arena.ofConfined()) {  
    MemorySegment memory =  
        arena.allocate(100);  
  
    // Use memory here  
}
```

Conceptually:





This makes the lifetime of native memory explicit rather than relying on Java's garbage collector to manage it directly.

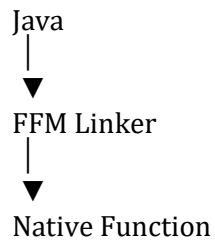
---

## 9. Upcalls and Downcalls

FFM supports two important directions of interaction.

### Downcall

Java → Native function

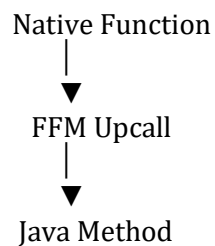


For example:

`strlen(...)`

### Upcall

Native code → Java method



This makes the API useful for more sophisticated native integrations where native libraries need to call back into Java.

---

## 10. FFM vs JNI

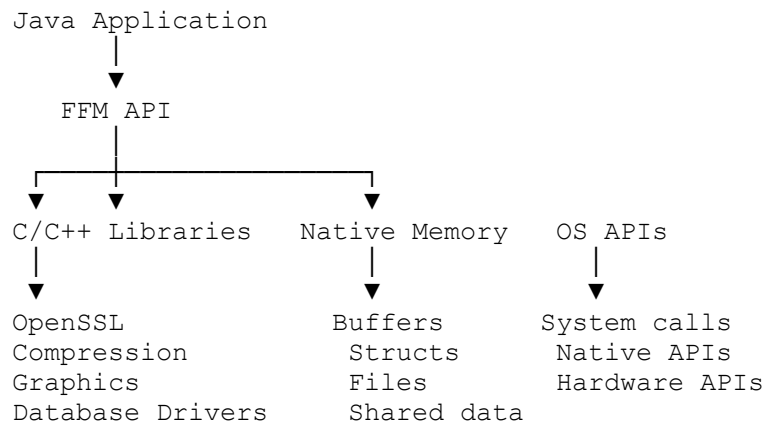
| Feature                      | JNI            | FFM API         |
|------------------------------|----------------|-----------------|
| Native interoperability      | Yes            | Yes             |
| Native memory access         | Yes            | Yes             |
| Requires native wrapper code | Often          | Usually no      |
| Type/layout descriptions     | More manual    | MemoryLayout    |
| Memory lifetime management   | More manual    | Arena           |
| Native function linking      | JNI mechanisms | Linker          |
| Java 21 standard API         | No             | Yes             |
| Safety/structured access     | Lower-level    | More structured |

FFM does **not** mean JNI disappears from Java overnight. Existing applications and libraries can continue using JNI, but FFM provides a modern alternative for many new native-interoperability use cases.

---

## 11. Real-World Use Cases

FFM is useful when Java needs to interact with:



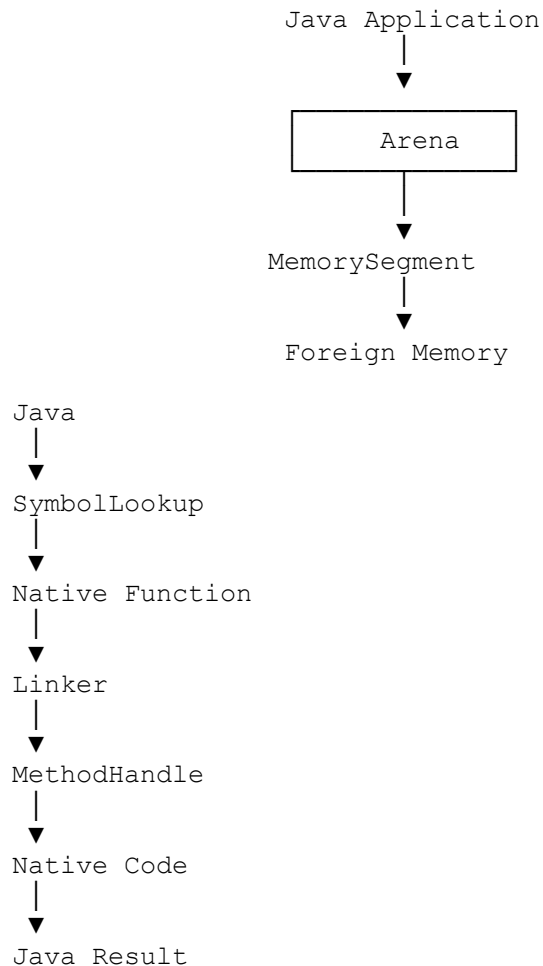
Typical use cases include:

- Calling existing C/C++ libraries
  - High-performance native-memory processing
  - Interoperating with operating-system APIs
  - Accessing native data structures
  - Integrating with native databases, compression, graphics, or scientific libraries
  - Reducing the need for custom JNI glue code
- 

## 12. Key Takeaway

**Java 21 finalized the Foreign Function and Memory API, providing a modern, structured way for Java applications to access foreign memory and call native functions without requiring traditional JNI wrappers for many use cases.**

The core architecture is:



**In one sentence:**

**The FFM API in Java 21 provides a safer, more expressive standard mechanism for Java code to work with off-heap memory and invoke functions implemented in native languages such as C and C++.**